Pratheek U B
289226

JavaScript Project

Create a module that exports a function and import it into another file.

Objective: This project demonstrates how to use JavaScript modules to import and export functions. Since OneCompiler does not support multiple files in Node.js, we will simulate module exports and imports within a single file.

Import and Use the Module

// Simulating script.js (Importing & Using the Module)

let length = 10;

let breadth = 5;

lib.area(length, breadth);

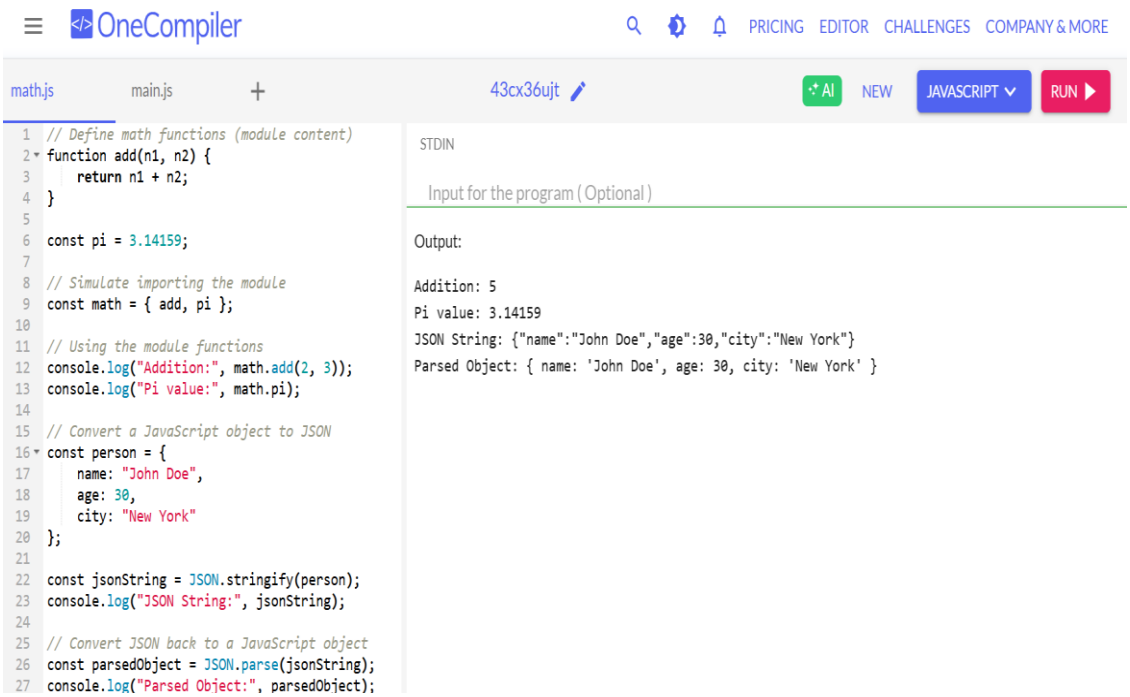
lib.perimeter(length, breadth);

Pratheek U B
289226

Convert a JavaScript object to JSON and back using JSON.stringify() and JSON.parse().

Objective: This project covers two key JavaScript concepts:

1. Converting a JavaScript Object to JSON and Back using JSON.stringify() and JSON.parse().
2. Including a JavaScript File in Another JavaScript File using ES6 Modules, CommonJS, and script tags.



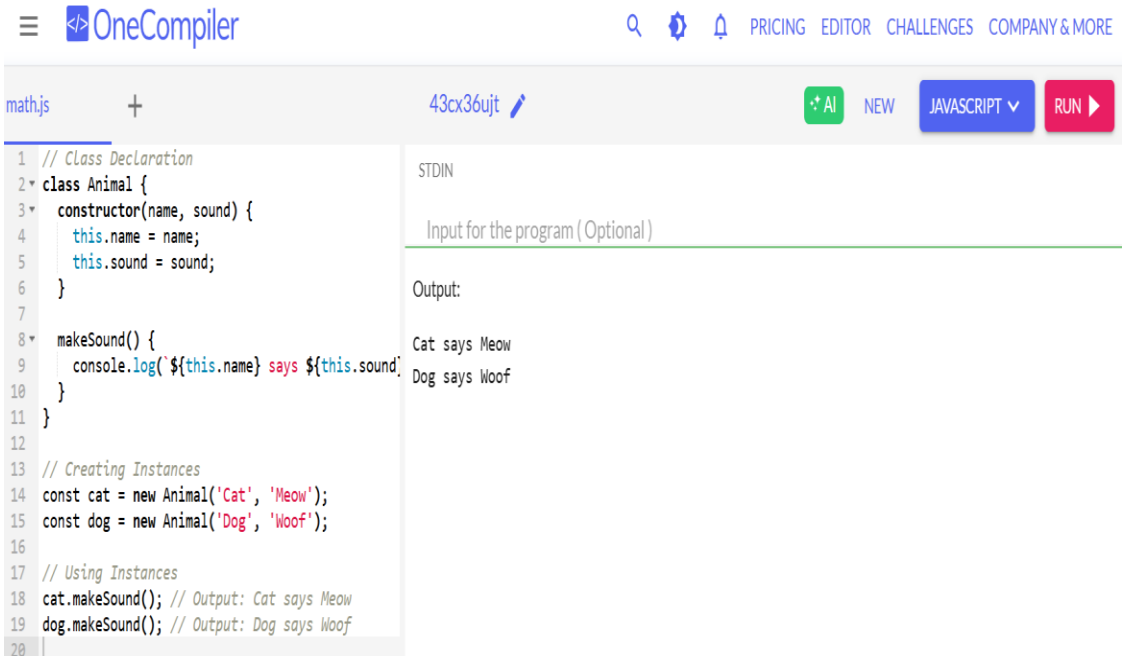
Conclusion summary:

- **Combines module and main file** since OneCompiler doesn't support multiple files.
- **Manually simulates module imports** using an object (math).
- **Ensures compatibility** by keeping everything in one script.

Define a class Animal with properties and a method, and create instances.

Objective: Create and use classes to define blueprints for creating objects with similar properties and behaviors.

Classes provide a way to implement object-oriented programming (OOP) concepts such as encapsulation, inheritance, and polymorphism.



Working conclusion:

- One file execution (OneCompiler doesn't support multiple files easily).
- Uses ES6 class syntax (supported in OneCompiler's Node.js runtime).
- Simple OOP demonstration with constructors and methods.

Write a promise that resolves after a delay and convert it into an async function using async/await.

Objective: The objective of this project is to demonstrate how to introduce a delay inside a loop in JavaScript using async/await with Promises. By implementing this, we can control the execution flow of asynchronous operations without blocking the main thread.

Conclusion Summary: By implementing async/await with Promises in loops, JavaScript allows controlled execution of asynchronous tasks without blocking the main thread. This technique is especially helpful when working with timers, animations, and API calls that require controlled delays.